

TECHNICAL DESIGN DOCUMENT

Gestionale Rendicontazione

Sistema di Rendicontazione Ore

v1.3.0 — Aggiornamento allo stato reale del backend + Roadmap Frontend Blazor

16 Luglio 2026

Documento riallineato al codice sorgente presente nel repository al momento della revisione.

Indice

Indice	2
1. Intestazione Tecnica	3
2. Architettura del Progetto	4
2.1 Struttura della Soluzione (.sln)	4
2.2 Modello Dati — Diagramma Entità-Relazione	4
2.3 Pattern e Principi Architettureali	6
3. Protocolli d'Utilizzo	8
3.1 Convenzioni Generali	8
3.2 Endpoint Implementati	8
3.2.1 Vista Admin	8
3.2.2 Vista Utente (User)	9
3.3 Esempi di Richiesta/Risposta	9
3.3.1 Login — POST /api/auth/login	9
3.3.2 Creazione WorkLog (utente) — POST /api/worklog	10
3.3.3 Report per progetto (admin) — GET /api/report-data/project/{id}	10
4. Visualizzazione Struttura Macro e Micro	11
4.1 Struttura Macro	11
4.2 Struttura Micro	11
4.2.1 Componenti del Layer API	11
4.2.2 Componenti del Layer DataAccess	11
4.2.3 Componenti del Layer Domain	12
4.2.4 Componenti del Frontend (Blazor WebAssembly)	12
4.2.5 Flusso di Generazione del Report PDF (disegno target)	12
5. Note Tecniche di Configurazione	14
5.1 Configurazione JWT (appsettings.json)	14
5.2 Connessione al database	14
5.3 Avvio, logging e seed dati	14
6. Stato di Avanzamento	15
6.1 Cosa è stato realizzato	15
6.2 Cosa manca rispetto al disegno originario	15
7. Roadmap per l'Avvio del Frontend Blazor WebAssembly	17
Obiettivo	17
Attività	17
Criterio di uscita	17
Obiettivo	17
Attività	17
Criterio di uscita	18
Obiettivo	18
Attività	18
Criterio di uscita	18
Obiettivo	18
Attività	19
Criterio di uscita	19

Obiettivo	19
Attività	19
Criterio di uscita	19
Obiettivo	20
Attività	20
Criterio di uscita	20
Obiettivo	20
Attività	20
Criterio di uscita	21
Riepilogo sequenza	21

1. Intestazione Tecnica

Nome Progetto	Gestionale Rendicontazione
Stato Documento	Aggiornato allo stato reale del codice — Backend completo per anagrafiche, autenticazione, WorkLog e reportistica. Frontend non ancora avviato.
Versione / Data	v1.3.0 / 16 Luglio 2026
Autori	Alessandro Gabriele Gregorio
Obiettivo Applicativo	Rendicontazione giornaliera e mensile delle attività lavorative dei tecnici, ai fini del controllo di gestione e della fatturazione, con generazione automatica di report in formato PDF.
Stack Backend	.NET 10, ASP.NET Core Web API, DevExpress XPO (ORM), SQLite (ambiente di sviluppo)
Stack Frontend	 Blazor WebAssembly (standalone), Blazorise Free/Community — da avviare (vedi Roadmap, cap. 7)
Libreria Generazione PDF	 PDFsharp (porting WASM), generazione lato client — da fare
Protocollo di Comunicazione	REST API JSON, documentate con Swagger (Swashbuckle) in ambiente di sviluppo
Autenticazione	JWT Bearer Token self-issued (HMAC-SHA256), con revoca lato server tramite blacklist su DB (vedi §2.3)

2. Architettura del Progetto

Il sistema adotta una variante pulita di Clean Architecture / DDD Light, finalizzata a isolare il nucleo di business (Domain) dalle dipendenze esterne di persistenza (DataAccess) e di esposizione (API). Il frontend Blazor WebAssembly comunicherà con il backend esclusivamente tramite chiamate HTTP/REST stateless: nessuna connessione persistente (SignalR/WebSocket) è prevista. Questo mantiene il backend privo di stato applicativo legato alla UI e semplifica la scalabilità orizzontale.

Rispetto alla revisione precedente del documento, il layer backend è oggi **sostanzialmente completo**: tutte le anagrafiche (Company, Project, Employee, Type, Status) espongono CRUD via REST, il modulo WorkLog distingue le viste Admin/Utente, ed è disponibile un endpoint di reportistica aggregata. Resta da avviare esclusivamente il progetto frontend, che non è ancora presente nella solution.

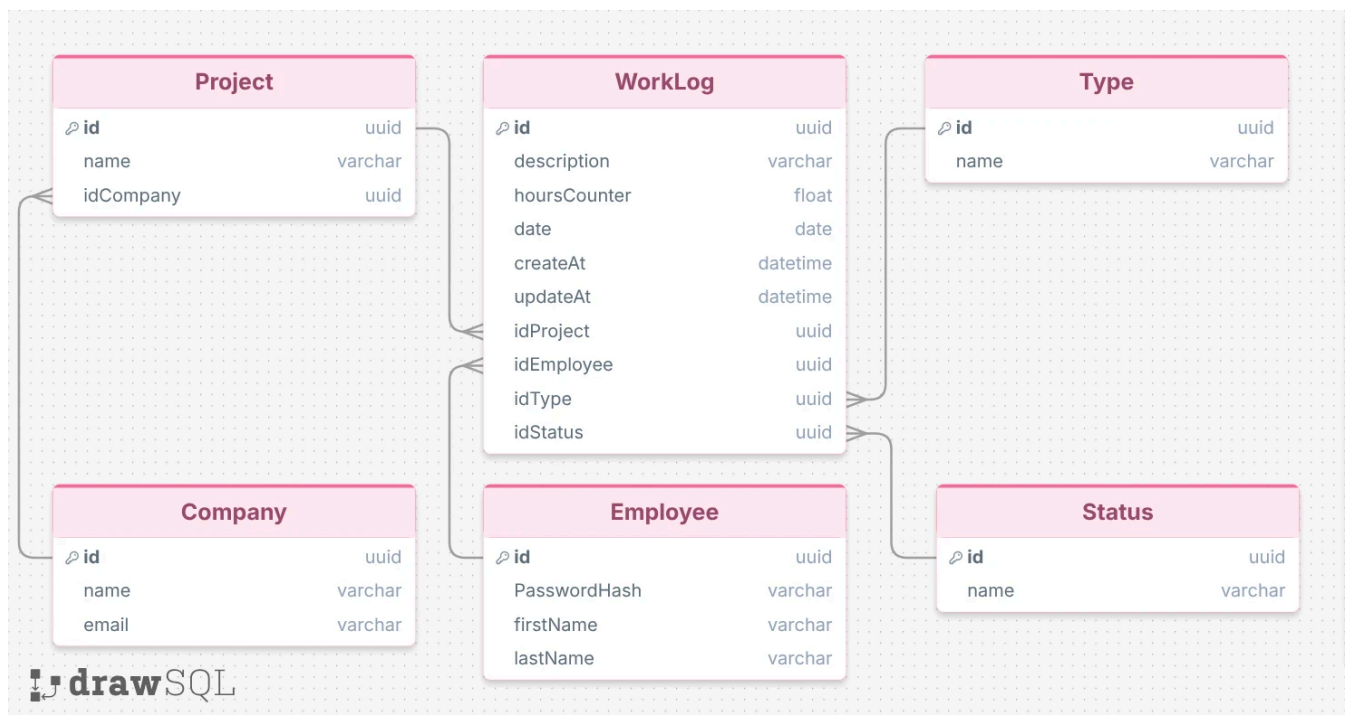
2.1 Struttura della Soluzione (.sln)

L'alberatura effettiva del codice sorgente backend, così come presente nel repository, è la seguente:

```
sln/
|
|-- API/                                # Layer di Esposizione (Presentazione Backend)
|   |-- Controllers/                    # 7 controller REST: Auth, Company, Employee, Project, Status, Type,
Worklog, ReportData
|   |-- Services/Jwt/                  # JwtOptions, JwtTokenService
|   |-- helpers/                      # ClaimsPrincipalExtensions, RequestContextEnricher, XpoSerilogSink
|   |-- Program.cs                    # Entry point, configurazione DI, middleware, seed
|
|-- DATAACCESS/                       # Layer di Infrastruttura e Persistenza
|   |-- Datacontext/                  # DbContextService (Session/UnitOfWork XPO), DataSeeder
|   |-- Services/                     # 10 servizi applicativi (Auth, Company, Employee, Project,
|                                   # Status, Type, Report, WorkLogAdmin, WorkLogUser, TokenBlacklist)
|   |-- helpers/                     # MappingProfile (AutoMapper), WorkLogQueryExtensions
|   |-- Manager/                     # predisposta, non ancora popolata
|
|-- DOMAIN/                           # Core Layer (isolato, zero dipendenze infrastrutturali)
|   |-- Entities/                    # Company, Project, Employee, Type, Status, WorkLog,
|                                   # BlacklistedToken, LogApplicativo
|   |-- Dtos/                        # Auth, Company, Employee, Project, Status, Type, WorkLog, Report
|   |-- Interfaces/                  # un contratto per ciascun servizio applicativo
|   |-- Constants/                  # RoleNames
```

2.2 Modello Dati — Diagramma Entità-Relazione

Il modello dati riflette il dominio applicativo della rendicontazione ore. L'entità centrale WorkLog rappresenta la singola riga di consuntivazione giornaliera, collegata a Project (e di conseguenza a Company), Employee, Type e Status. La struttura delle relazioni è confermata dal codice attuale (attributi [Association] di XPO) e non è cambiata rispetto alla revisione precedente.



Company	
<i>Anagrafica società cliente/interna. Relazione 1-N con Project tramite Project.Company (colonna idCompany).</i>	
Campo	Tipo / Note
id	Guid — chiave primaria XPO (autogenerata)
name	varchar, indice univoco
email	varchar, indice univoco

Project	
<i>Progetto associato a una Company. Relazione N-1 con Company (idCompany) e 1-N con WorkLog (idProject).</i>	
Campo	Tipo / Note
id	Guid — chiave primaria XPO
name	varchar
idCompany	FK → Company

Employee	
<i>Tecnico/dipendente che rendiconta le ore. Relazione 1-N con WorkLog tramite WorkLog.Employee (idEmployee). Estende PermissionPolicyUser di DevExpress (ereditandone Oid come Id, UserName, IsActive, Roles) per anticipare l'autorizzazione a ruoli</i>	
Campo	Tipo / Note
id	Guid — alias di Oid, ereditato da PermissionPolicyUser
passwordHash	varchar(1024) — hash generato con Microsoft.AspNetCore.Identity.PasswordHasher
firstName / lastName	varchar
Roles	collection ereditata — oggi valorizzata con i ruoli Admin/User

Type	
<i>Tipologia di attività rendicontata (es. FIX). Relazione 1-N con WorkLog tramite WorkLog.Type (idType).</i>	
Campo	Tipo / Note
id	Guid — chiave primaria XPO
name	varchar, indice univoco

Status

Stato della rendicontazione (es. *WORKING_PROGRESS*, *REJECTED*). Relazione 1-N con *WorkLog* tramite *WorkLog.Status* (*idStatus*).

Campo	Tipo / Note
id	Guid — chiave primaria XPO
name	varchar, indice univoco

WorkLog

Entità centrale del modello: singola riga di rendicontazione ore. Relazione N-1 con *Project*, *Employee*, *Type* e *Status*.

Campo	Tipo / Note
id	Guid — chiave primaria XPO
description	varchar
hoursCounter	float, vincolato 1–24 a livello di DTO
date	DateOnly
createAt / updateAt	datetime, gestiti dal server
isDeleted	bool — soft delete (nuovo rispetto alla v1.2.0)
idProject / idEmployee / idType / idStatus	FK

BlacklistedToken

Nuova entità (non presente nello schema): registro dei JWT invalidati lato server al logout.

Campo	Tipo / Note
jti	string — chiave primaria (claim jti del token)
expiresAt	datetime — usato per la potatura automatica dei record scaduti

2.3 Pattern e Principi Architeturali

Il layer DataAccess espone un servizio unico, IDbContextService, che incapsula due modalità di accesso al database XPO:

- **Sola lettura** — `ExecuteReadOnly<T>(Func<Session,T>) / ExecuteReadOnly(Action<Session>)`: apre una Session XPO con `TrackPropertiesModifications = false`, eseguita in modo sincrono.
- **Lettura e scrittura** — `ReadWrite(Func<UnitOfWork,Task>)` e le varianti `ReadWriteAsync` con propagazione del `CancellationToken`: apre una `UnitOfWork` XPO, esegue l'operazione e chiama `CommitChangesAsync()` al termine; in caso di eccezione la modifica non viene salvata.

Il `IDataLayer` XPO (`ThreadSafeDataLayer`) è registrato come Singleton in `AddXpoInfrastructure`, mentre `IDbContextService` è Scoped — uno per richiesta HTTP. Tutte le dipendenze applicative sono registrate nel container nativo di ASP.NET Core in `Program.cs`, inclusi i dieci servizi applicativi elencati in §2.1.

- **DTO Mapping** — nessuna Entity è esposta direttamente verso l'esterno. Il mapping Entity → DTO è centralizzato in `MappingProfile` (`AutoMapper`), registrato in `Program.cs`.
- **Validazione** — Data Annotations sui Dtos in ingresso (`[Required]`, `[Range]`, `[StringLength]`), verificate tramite `ModelState.IsValid` nel Controller.
- **Autorizzazione a ruoli** — `CompanyController` ed `EmployeeController` richiedono `[Authorize(Roles = "Admin")]` a livello di classe; `ProjectController`, `StatusController`, `TypeController` distinguono le singole azioni (lettura autenticata, scrittura riservata all'Admin); `WorklogController` è un controller unico che internamente instrada la richiesta al servizio Admin o Utente in base al ruolo (`User.IsInRole`).
- **Revoca token (logout)** — `TokenBlacklistService` persiste il claim dei token invalidati su DB (entità `BlacklistedToken`) con potatura asincrona dei record scaduti. La validazione JWT (`JwtBearerEvents.OnTokenValidated`) consulta la blacklist a ogni richiesta autenticata: i JWT self-issued restano stateless nella generazione, ma la revoca è ora effettiva lato server.
- **Logging strutturato** — Serilog con due sink, Console e un sink custom (`XpoSerilogSink`) che scrive sulla tabella `LogApplicativo` via XPO, arricchito con `RequestContextEnricher` (metodo/path della richiesta corrente).

PARZIALE Gestione errori centralizzata — è presente un handler in `Program.cs` (`app.UseExceptionHandler(...)`) che mappa le eccezioni note (`InvalidOperationException`, `UnauthorizedAccessException`, `ArgumentException`) a codici HTTP specifici e restituisce un `ProblemDetails` coerente, loggando l'eccezione. Non è però ancora estratto in una classe `ExceptionHandlingMiddleware` dedicata e riutilizzabile: la logica vive come lambda inline in `Program.cs`.

DA FARE Separation of Concerns per la generazione PDF — la generazione del report (per tecnico o per progetto) è prevista interamente a carico del frontend Blazor WebAssembly (componente `PdfReportBuilder`); il backend espone già i dati aggregati in JSON (`ReportDataController`, vedi §3), ma nessun componente frontend esiste ancora.

3. Protocolli d'Utilizzo

La comunicazione tra Frontend e Backend avviene tramite API RESTful, con payload in formato JSON. Il backend è raggiungibile ed esplorabile anche tramite Swagger UI (Swashbuckle), attivo in ambiente di sviluppo e configurato con autenticazione Bearer integrata nell'interfaccia.

3.1 Convenzioni Generali

- **Versionamento** — tutti gli endpoint sono esposti sotto il prefisso `/api/`.
- **Formato dati** — tutte le risposte del backend sono in formato `application/json`.
- **Autenticazione** — JWT Bearer Token self-issued (HMAC-SHA256), inserito nell'header `Authorization` su ogni richiesta autenticata. Scadenza configurabile (`Jwt:ExpiryMinutes`, default 60 minuti). Ogni token è verificato anche contro la blacklist server-side al momento della validazione (vedi §2.3).
- **Autorizzazione** — due ruoli applicativi, Admin e User (`Domain.Constants.RoleNames`). Gli endpoint di anagrafica scrittura sono riservati all'Admin; il modulo WorkLog è condiviso con instradamento interno per ruolo.
- **Codici di stato HTTP** — 200 OK, 201 Created, 204 No Content, 400 Bad Request/ValidationProblem, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable Entity (es. registrazione con `userName` duplicato), 500 Internal Server Error. Tutti effettivamente esercitati dagli endpoint oggi implementati.
- **Gestione errori** — risposta in formato `ProblemDetails` (RFC 7807), sia tramite `Problem()/ValidationProblem()` nei singoli controller sia tramite l'handler centralizzato in `Program.cs` per le eccezioni non gestite (vedi §2.3).
- **Documentazione interattiva** — Swagger / OpenAPI (Swashbuckle.AspNetCore) in ambiente di sviluppo.

3.2 Endpoint Implementati

A differenza della revisione precedente del documento, tutti gli endpoint elencati di seguito sono implementati, testati manualmente (vedi +DOCUMENTAZIONE/Curl.md) e coperti da `[Authorize]` con la granularità indicata.

3.2.2 Vista Utente (User)

Metodo	Endpoint	Descrizione
POST	<code>/api/auth/login</code>	Verifica <code>userName/password</code> , restituisce token JWT
POST	<code>/api/auth/logout</code>	Invalida il proprio token
GET/POST/PUT/DELETE	<code>/api/worklog</code>	CRUD sui propri worklog: <code>IdEmployee</code> sempre dedotto dal token, mai dal body
GET	<code>/api/project</code>	Elenco progetti (sola lettura)
GET	<code>/api/type</code>	Elenco Type (sola lettura)
GET	<code>/api/status</code>	Elenco Status (sola lettura)
GET	<code>/api/report-data/employee</code>	Report aggregato delle proprie rendicontazioni/periodo

3.2.1 Vista Admin

Metodo	Endpoint	Descrizione
POST	/api/auth/register	Crea un nuovo Employee (solo Admin)
POST	/api/auth/login	Verifica userName/password, restituisce token JWT + scadenza + displayName
POST	/api/auth/logout	Invalida il token corrente (jti) inserendolo nella blacklist su DB
GET/POST/PUT/DELETE	/api/company	CRUD completo anagrafica Company
GET/PUT/DELETE	/api/employee	Consultazione, aggiornamento ed eliminazione Employee
GET/POST/PUT/DELETE	/api/project	CRUD completo anagrafica Project (lettura anche per utente autenticato)
GET/POST/PUT/DELETE	/api/type	CRUD completo Type (lettura anche per utente autenticato)
GET/POST/PUT/DELETE	/api/status	CRUD completo Status (lettura anche per utente autenticato)
GET/POST/PUT/DELETE	/api/worklog	CRUD su tutti i worklog di tutti i dipendenti, con filtri (employee, project, periodo, status)
GET	/api/report-data/project/{id}	Report aggregato per progetto/periodo, con dettaglio per dipendente e worklog completo
GET	/api/report-data/employee/{id}	Report aggregato per dipendente/periodo, con dettaglio per progetto

4. Visualizzazione Struttura Macro e Micro

4.1 Struttura Macro

A livello macro il sistema si compone di tre macro-blocchi applicativi:

-  **Backend API** (.NET 10 / ASP.NET Core) — espone tutti gli endpoint REST descritti al capitolo 3 e applica la logica di business e di autorizzazione.
-  **Livello Dati** — SQLite in ambiente di sviluppo. PostgreSQL/SQL Server per la produzione restano un'opzione aperta: grazie a XPO il cambio di provider richiede solo una nuova connection string.
-  **Frontend Blazor WebAssembly** — interfaccia utente per inserimento ore, consultazione dati e composizione/download del report PDF in locale. Non ancora presente nel repository: vedi Roadmap, capitolo 7.

4.2 Struttura Micro

4.2.1 Componenti del Layer API

- AuthController, CompanyController, EmployeeController, ProjectController, StatusController, TypeController, WorklogController, ReportDataController — 8 controller REST, tutti operativi.

✓ Fatto

- JwtTokenService — crea il token JWT (HMAC-SHA256) a partire dai claim costruiti da AuthService.

✓ Fatto

- Gestione centralizzata delle eccezioni — presente come handler inline in Program.cs (mapping eccezione → status code → ProblemDetails), non ancora estratta in una classe dedicata riutilizzabile.

● Parziale

4.2.2 Componenti del Layer DataAccess

- DbContextService — implementa IDbContextService, incapsula Session (sola lettura) e UnitOfWork (lettura/scrittura) di XPO.

✓ Fatto

- AuthService — login con hashing/re-hash password, costruzione claim (incluso ruolo) e generazione token; RegisterAsync per la creazione di nuovi Employee da parte dell'Admin.

✓ Fatto

- CompanyService, EmployeeService, ProjectService, StatusService, TypeService — CRUD completo per le rispettive anagrafiche, con mapping via AutoMapper.

✓ Fatto

- WorkLogAdminService / WorkLogUserService — CRUD e filtri separati per le due viste (admin su tutti i dipendenti, utente sui propri record), con soft delete tramite il campo IsWorkLogDeleted.

✓ Fatto

- ReportService — aggregazione dati per progetto e per dipendente (totale ore, ripartizione per tipo/stato/giorno/progetto/dipendente).

✓ Fatto

- TokenBlacklistService — persistenza e verifica dei token invalidati al logout, con potatura asincrona dei record scaduti.

✓ Fatto

- DataSeeder — popola il database al primo avvio con dati demo e un utente admin di default, in modo idempotente.

✓ Fatto

4.2.3 Componenti del Layer Domain

- Entities — Company, Project, Employee, Type, Status, WorkLog, BlacklistedToken, LogApplicativo: tutte presenti e mappate su XPO.

✓ Fatto

- Dtos — presenti per tutte le anagrafiche, l'autenticazione, il modulo WorkLog (viste Admin/User) e la reportistica.

 Fatto

- Interfaces — un contratto per ciascun servizio applicativo (10 interfacce + IDbContextService).

 Fatto

4.2.4 Componenti del Frontend (Blazor WebAssembly)

Nessuno dei componenti seguenti è ancora presente nel repository: la solution contiene solo i progetti backend. Restano descritti come disegno target, dettagliato in forma di roadmap operativa al capitolo 7.

- WorkLogReportPage — pagina Razor per la scelta del tipo di report, dell'entità e del periodo, e il download del PDF.
- ReportApiClient — wrapper su HttpClient tipizzato verso gli endpoint /api/report-data/*.
- PdfReportBuilder — servizio C# in WASM che compone il PDF con PDFsharp a partire dai DTO di report.
- Modulo JS interop (download.js) — espone il blob generato in memoria come download verso il filesystem dell'utente.

 Pianificato — vedi capitolo 7

4.2.5 Flusso di Generazione del Report PDF (disegno target)

In termini semplici, il programma dovrà restituire un report in formato PDF che fotografa lo stato di avanzamento del lavoro svolto in un determinato periodo, in due varianti selezionabili dall'utente: report per tecnico e report per progetto. Il backend fornisce già i dati aggregati necessari (§3.3.3); la generazione del documento sarà interamente a carico del client.

1. L'utente, dalla pagina WorkLogReportPage, sceglie tipologia, entità (tecnico o progetto) e periodo, e richiede il download.
2. Il ReportApiClient invoca GET /api/report-data/project/{id} oppure GET /api/report-data/employee/{id} (o l'equivalente /api/report-data/employee per l'utente corrente), allegando il token JWT.
3. Il ReportDataController valida i parametri (incluso l'intervallo massimo di un anno) e invoca IReportService.
4. Il backend restituisce i dati aggregati in JSON — già implementato e verificato.
5. PdfReportBuilder, eseguito in WASM nel browser, compone il PDF (intestazione, tabella ore, riepilogo, firma/data) tramite PDFsharp.
6. Il PDF, generato in memoria, viene esposto come download diretto tramite il modulo JS interop, senza ulteriori chiamate al backend.

Questa scelta architetturale elimina la dipendenza da una libreria di rendering PDF lato server, mantenendo il backend come puro fornitore di dati stateless. Qualora in futuro si rendesse necessario produrre il PDF anche in assenza di un client (es. invio email automatico schedato, archiviazione lato server), sarà necessario reintrodurre una generazione equivalente lato backend.

5. Note Tecniche di Configurazione

5.1 Configurazione JWT (appsettings.json)

```
"Jwt": {  
  "Issuer": "GestionaleRendicontazione.Api",  
  "Audience": "GestionaleRendicontazione.Client",  
  "SecretKey": "<almeno 32 caratteri>",  
  "ExpiryMinutes": 60  
}
```

Attenzione: il repository contiene una SecretKey segnaposto ("CAMBIA-QUESTA-CHIAVE-IN-UN-SEGRETO-LUNGO-ALMENO-32-CARATTERI"). Resta in cima alla lista degli interventi di hardening pre-produzione (§6.2).

5.2 Connessione al database

```
"ConnectionStrings": {  
  "DefaultConnection": "XpoProvider=SQLite;Data Source=rendicontazione.db;"  
}
```

Lo schema del database viene creato/aggiornato automaticamente all'avvio (AutoCreateOption.DatabaseAndSchema) sulla base delle Entities mappate per riflessione, all'avvio viene inoltre forzato il WAL mode di SQLite (PRAGMA journal_mode=WAL), per ridurre il rischio di errori "database is locked" quando più richieste concorrenti arrivano insieme.

5.3 Avvio, logging e seed dati

All'avvio dell'applicazione (Program.cs) viene invocato DataSeeder.SeedAsync, che popola in modo idempotente dati demo (Company, Project, Type, due Status, un WorkLog) e crea un utente admin (userName admin, password Admin123!) se non già presente. Da disabilitare prima di un deploy in produzione (§6.2).

Il logging è gestito da Serilog, con output su Console e su tabella DB (LogApplicativo) tramite un sink custom (XpoSerilogSink), arricchito con il metodo e il path della richiesta HTTP corrente.

6. Stato di Avanzamento

Sintesi dello stato reale del progetto alla data di questa revisione. La roadmap operativa per il frontend è sviluppata a parte, al capitolo 7.

6.1 Cosa è stato realizzato

Area	Dettaglio
Struttura solution	3 progetti (API, DATAACCESS, DOMAIN) con dipendenze corrette verso l'interno (Domain isolato)
Persistenza	DevExpress XPO su SQLite in WAL mode, schema autogenerato per riflessione, pattern Session/UnitOfWork incapsulato in IDbContextService
Modello dati	Entities Company, Project, Employee, Type, Status, WorkLog, BlacklistedToken, LogApplicativo, con relazioni XPO e soft delete su WorkLog
Autenticazione e autorizzazione	Login/logout JWT self-issued (HMAC-SHA256) con revoca lato server (blacklist su DB), hashing password, autorizzazione a ruoli (Admin/User) effettivamente applicata sui controller
API di dominio	CRUD completo per Company, Employee, Project, Type, Status; modulo WorkLog con viste separate Admin/Utente
Reportistica	Endpoint di aggregazione dati per progetto e per dipendente (ReportDataController + IReportService), pronti per il consumo dal frontend
Mapping	AutoMapper con profilo centralizzato (MappingProfile), in sostituzione del mapping manuale
Logging	Serilog strutturato su Console e su DB, con contesto di richiesta
Gestione errori	Handler centralizzato in Program.cs con risposte ProblemDetails coerenti
Seed dati	DataSeeder idempotente con dati demo e utente admin
Documentazione API	Swagger/OpenAPI in ambiente di sviluppo (con supporto Bearer); DOCUMENTAZIONE/Curl.md con esempi curl per tutti i moduli

6.2 Cosa manca rispetto al disegno originario

Area	Dettaglio
Frontend	Nessun progetto Blazor WebAssembly nel repository — vedi Roadmap, capitolo 7
Generazione PDF	PdfReportBuilder / PDFsharp / modulo JS interop non implementati (dipendono dal frontend)
Middleware errori dedicato	La gestione centralizzata esiste ma è una lambda inline in Program.cs, non una classe ExceptionHandlingMiddleware riutilizzabile e testabile in isolamento
Sicurezza configurazione	SecretKey JWT segnaposto nel repository; nessuna gestione differenziata per ambiente di produzione (secret manager / variabili d'ambiente)
Ambiente di produzione	DataSeeder da rendere condizionale all'ambiente; database di produzione (PostgreSQL/SQL Server) da scegliere e validare in staging
Test automatici	Nessun progetto di test (unit test sui Services, integration test sugli endpoint)
DATAACCESS/Manager	Cartella predisposta per orchestrazioni cross-entità, ancora vuota

7. Roadmap per l'Avvio del Frontend Blazor WebAssembly

Il backend espone oggi tutte le API necessarie (anagrafiche, WorkLog, autenticazione a ruoli, reportistica aggregata): è la condizione abilitante per partire con il frontend. Questa roadmap traduce il disegno target descritto al §4.2.4 in una sequenza di fasi concrete, pensate per essere eseguite in ordine e ciascuna verificabile.

Premessa tecnologica: Blazor WebAssembly standalone (.NET 10, coerente con il backend), Blazorise Free/Community come libreria di componenti UI, comunicazione esclusivamente REST/JSON verso l'API esistente. Nessuna dipendenza da SignalR.

Fase F1 — Bootstrap del progetto	<i>0.5–1 giorno</i>
---	---------------------

Obiettivo

Un progetto Blazor WASM che compila, gira e viene aggiunto alla solution esistente, con la UI base di Blazorise già visibile.

Attività

- Creare il progetto FRONTEND/GestionaleRendicontazione.Client (Blazor WebAssembly standalone, .NET 10) e aggiungerlo alla .sln accanto ad API, DATAACCESS, DOMAIN.
- Installare Blazorise.Bootstrap5 (o Blazorise.FluentUI2) + Blazorise.Icons.FontAwesome, registrare i servizi in Program.cs del client.
- Configurare wwwroot/appsettings.json e appsettings.Development.json con la base URL dell'API (es. https://localhost:5182), letta da un HttpClient tipizzato registrato in DI.
- Impostare CORS sul backend (oggi assente) per accettare richieste dall'origine del client in sviluppo.
- Struttura cartelle: Pages/, Layout/, Services/ (ApiClient), Models/ (DTO condivisi o duplicati lato client), wwwroot/.

Criterio di uscita

dotnet run sul client mostra una pagina Blazorise di base che effettua una chiamata anonima riuscita verso un endpoint pubblico del backend (es. Swagger raggiungibile, o un healthcheck).

Fase F2 — Autenticazione e gestione sessione

1-2 giorni

Obiettivo

Login funzionante, token persistito e allegato automaticamente a ogni chiamata, logout che invalida il token anche lato server.

Attività

- Pagina LoginPage con form (userName/password) via Blazorise, che chiama POST /api/auth/login tramite un AuthApiClient dedicato.
- Persistenza del token con Blazored.LocalStorage (o SessionStorage se si preferisce non sopravvivere alla chiusura del browser).
- Un DelegatingHandler custom (AuthorizationMessageHandler) che allega automaticamente l'header Authorization: Bearer <token> a ogni HttpClient tipizzato.
- Un CustomAuthenticationStateProvider che decodifica il JWT (claim role, name, exp) per popolare ClaimsPrincipal, usato da <AuthorizeView> e <CascadingAuthenticationState> nel layout.
- Gestione della scadenza: intercettare le risposte 401 nel DelegatingHandler, forzare logout locale e redirect a /login.
- LogoutButton che chiama POST /api/auth/logout (per la revoca server-side) e solo dopo pulisce lo storage locale.

Criterio di uscita

Login con l'utente admin seed (admin / Admin123!) funzionante end-to-end; una chiamata autenticata verso un endpoint protetto (es. GET /api/company) va a buon fine; il logout invalida realmente il token (una chiamata successiva con lo stesso token restituisce 401).

Fase F3 — Layout, navigazione e routing per ruolo

1 giorno

Obiettivo

Un'ossatura di navigazione coerente con i due ruoli applicativi (Admin/User), pronta a ospitare le pagine funzionali.

Attività

- MainLayout con Blazorise Bar/Sidebar, NavMenu con voci condizionate da <AuthorizeView Roles="Admin"> per le sezioni di anagrafica.
- Route protette con [Authorize] sui componenti pagina e un componente RedirectToLogin per gli accessi anonimi.
- Pagina 403/Forbidden e pagina 404 dedicate, invece del comportamento di default di Blazor.
- Convenzione di progetto per i DTO condivisi con il backend (o generazione automatica da OpenAPI, valutare NSwag/Kiota per evitare la duplicazione manuale).

Criterio di uscita

Un utente con ruolo User non vede né può raggiungere via URL diretto le pagine di anagrafica riservate all'Admin.

Fase F4 — Moduli di anagrafica (Admin)

2–3 giorni

Obiettivo

CRUD completo, da interfaccia, per Company, Project, Employee, Type, Status — a consumo diretto delle API già disponibili.

Attività

- Un ApiClient tipizzato per ciascuna risorsa (CompanyApiClient, ProjectApiClient, ...), con i metodi GetAll/GetById/Create/Update/Delete.
- Pagine elenco con Blazorise DataGrid (paginazione, ordinamento, filtro testuale lato client per i volumi contenuti attesi).
- Modali o pagine dedicate per Create/Edit, con Blazorise Validation collegata alle stesse regole già presenti nei Dtos (Required, Range, ecc.), per un feedback immediato prima della submit.
- Gestione centralizzata degli errori di validazione 400/ValidationProblem restituiti dal backend, mappati sui campi del form.
- Project collegato a Company tramite una select popolata da GET /api/company.

Criterio di uscita

Un Admin può creare un'azienda, un progetto collegato, un Type e uno Status, e vederli comparire nelle rispettive liste senza ricaricare manualmente la pagina.

Fase F5 — Modulo WorkLog (inserimento e consultazione ore)

2–3 giorni

Obiettivo

Il cuore applicativo per l'utente finale: inserire, consultare, modificare ed eliminare le proprie rendicontazioni; per l'Admin, la vista completa con filtri.

Attività

- WorkLogApiClient che consuma GET/POST/PUT/DELETE /api/worklog, adattando il payload alla vista Admin o User (il backend instrada già internamente, ma il client deve comunque non inviare mai IdEmployee per l'utente non Admin).
- Pagina MyWorkLogsPage (utente): elenco delle proprie rendicontazioni, form di inserimento rapido (data, ore, descrizione, project/type/status da select), modifica ed eliminazione inline.
- Pagina AdminWorkLogsPage: stessa vista con filtri aggiuntivi (dipendente, progetto, periodo, stato), riservata al ruolo Admin.

- Validazione client-side coerente con i vincoli server (ore tra 1 e 24, campi obbligatori) per ridurre i round-trip falliti.
- Gestione ottimistica o pessimistica degli aggiornamenti in lista dopo Create/Update/Delete (scegliere una strategia e mantenerla coerente in tutto il modulo).

Criterio di uscita

Un utente User inserisce una rendicontazione di 8 ore su un progetto esistente e la ritrova nel proprio elenco; un Admin la vede comparire anche nella vista aggregata con i filtri applicati.

Fase F6 — Reportistica e generazione PDF lato client

2–4 giorni

Obiettivo

Chiudere il disegno target descritto al §4.2.5: selezione del report, download di un PDF generato interamente nel browser.

Attività

- ReportApiClient verso GET /api/report-data/project/{id} e GET /api/report-data/employee/{id} (o /employee per l'utente corrente).
- WorkLogReportPage: selezione tipo report (per progetto / per tecnico), entità e intervallo di date (rispettando il vincolo di un anno massimo già validato dal backend).
- Integrazione di una libreria di generazione PDF compatibile con WASM (PDFsharp via porting, oppure valutare in alternativa QuestPDF se compatibile con il target WASM al momento dell'implementazione: verificarne la licenza per uso commerciale).
- Componente PdfReportBuilder che compone intestazione, tabella ore, riepilogo per tipo/stato/progetto o dipendente, e firma/data, a partire dai DTO di report già disponibili.
- Modulo JS interop (download.js) per esporre il blob generato in memoria come download diretto, senza ulteriori chiamate al backend.

Criterio di uscita

Download di un PDF leggibile sia per un report "per progetto" sia per un report "per tecnico", su un intervallo di date reale con dati seed.

Fase F7 — Hardening e preparazione al rilascio

1–2 giorni

Obiettivo

Portare il frontend da "funzionante in locale" a "pronto per un ambiente condiviso".

Attività

- Gestione centralizzata degli errori di rete/timeout con notifiche Blazorise (Snackbar/Toast) invece di eccezioni non gestite in console.

- Configurazione multi-ambiente (appsettings.Development.json vs appsettings.Production.json) per la base URL dell'API.
- Build di produzione (dotnet publish -c Release) con AOT/trimming valutati in base ai tempi di primo caricamento accettabili.
- Hosting statico (il client compilato è un set di file statici: IIS, Nginx, Azure Static Web Apps o storage con CDN sono tutte opzioni compatibili) con configurazione del fallback SPA (tutte le route non-file rediretto a index.html).
- Allineare l'hardening lato backend descritto al §6.2 (SecretKey JWT gestita per ambiente, CORS ristretto all'origine di produzione del client) prima di esporre l'accoppiata frontend+backend fuori dalla rete locale.

Criterio di uscita

Build di produzione del client servita da un host statico, puntata contro un'istanza dell'API con configurazione di produzione, login e flusso WorkLog completo funzionanti end-to-end.

Riepilogo sequenza

Fase	Contenuto	Precondizione
F1	Bootstrap progetto Blazor WASM + Blazorise	—
F2	Autenticazione, token, logout con revoca	F1
F3	Layout, navigazione, routing per ruolo	F2
F4	CRUD anagrafiche (Admin)	F3
F5	Modulo WorkLog (utente + admin)	F3
F6	Reportistica + PDF lato client	F4, F5
F7	Hardening e rilascio	F6

Nota: F4 e F5 non hanno dipendenza reciproca stretta e possono procedere in parallelo se sul progetto lavora più di una persona; F6 richiede che entrambe siano concluse perché il report "per progetto" include il dettaglio per dipendente e il report "per tecnico" include il dettaglio per progetto.